

第七章 多无人机 Agent 协同

6.1 多 Agent 系统简介

6.1.1 从单机到集群

单个无人机的能力终究有限，而由多个无人机组成的集群或“蜂群”则能产生力量倍增的效应。多无人机系统的出现，源于单一平台在覆盖范围、任务效率和系统鲁棒性方面的固有局限性。协同作业的需求推动了从单机智能到集群智能的演进。



1 基于大模型 Agent 的集群协作

本章就是将基于大型语言模型（LLM）的智能体（Agent）作为这些多无人机系统的认知核心。我们将探索实现这种协作的架构模式，通过一个动手实验来验证——您将在高保真度的 AirSim 模拟器中构建并部署一个多无人机智能体系统。

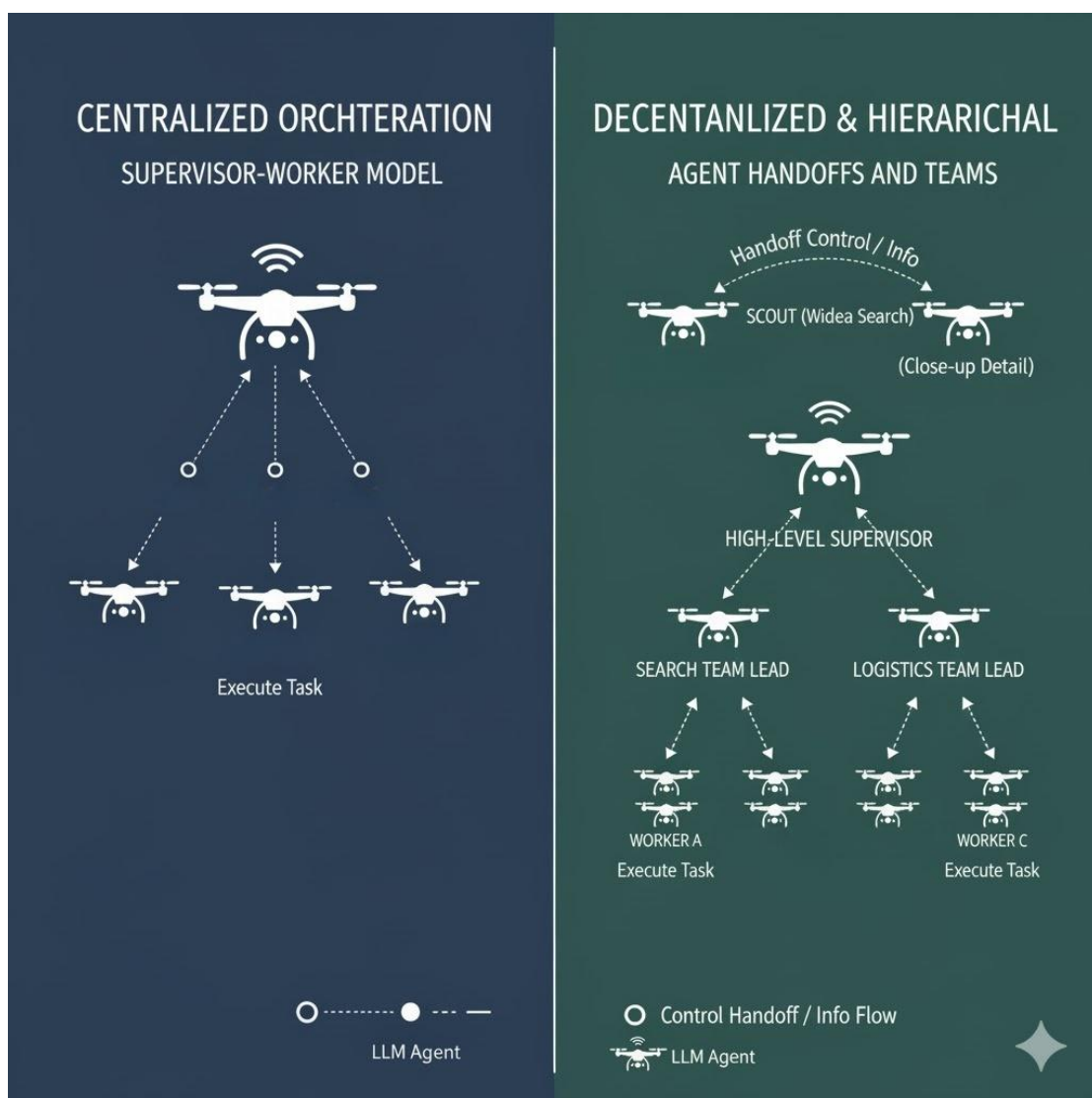
但任何多智能体系统，无论是无人机集群还是分布式计算网络，都必须解决一系列根本性的挑战：

- **任务分配与分解 (Task Allocation and Decomposition):** 如何将一个高层级的任务目标（例如，“搜索这片公园内的失踪徒步者”）分解为一系列可执行的子任务（例如，“无人机 1 搜索 A 区”、“无人机 2 搜索 B 区”），并如何将这子任务分配给最合适的智能体？
- **通信 (Communication):** 智能体之间需要共享哪些信息？如何构建通信协议，使其既高效又无歧义？
- **去中心化决策与共识 (Decentralized Decision-Making and Consensus):** 在没有一个全知全能的中央控制器的情况下，智能体如何基于不完整的信息做出决策？一个集群如何达成集体共识或调整其整体策略？

- **路径规划与冲突消解 (Path Planning and Deconfliction):** 这是确保系统物理安全的关键。每架无人机都需要规划出到达其目标的路径，同时必须避免与集群中的其他无人机以及环境中的障碍物发生碰撞。

2 多智能体协同模式

将多个独立的 LLM Agent 连接起来，形成一个能够协同工作的系统，需要精心设计的架构。这些架构或“设计模式”决定了智能体之间的信息流和控制关系，是影响系统效率、可扩展性和复杂性的关键设计决策。



3 监督者-工作者模型（集中式编排）

- **概念:** 这是最常见也是最直观的协同模式。一个“监督者”（**Supervisor**）智能体接收来自人类操作员的最高层级任务目标。它负责进行宏观的任务分解和规划，然后将具体的子任务委派给多个“工作者”（**Worker**）智能体。每个工作者智能体通常控制一架无人机，专注于执行分配给它的具体任务（如飞往某个区域、采集数据等）。工作者完成任务后向监督者汇报结果，监督者则负责汇总信息、评估任务进展，并规划下一步的行动。

- 框架实现（以 LangGraph 为例）：这种模式可以非常自然地用图（Graph）结构来表示。监督者是图中的一个核心节点，它根据当前的系统状态（State）将任务路由到其他代表工作者的节点 11。在 LangGraph 框架中，我们可以使用一个中央路由器或条件边（Conditional Edge）逻辑来实现监督者的决策过程，决定接下来激活哪个工作者。这种架构非常适合那些可以被清晰地分解为多个并行子任务的场景，例如区域搜索或大规模测绘。

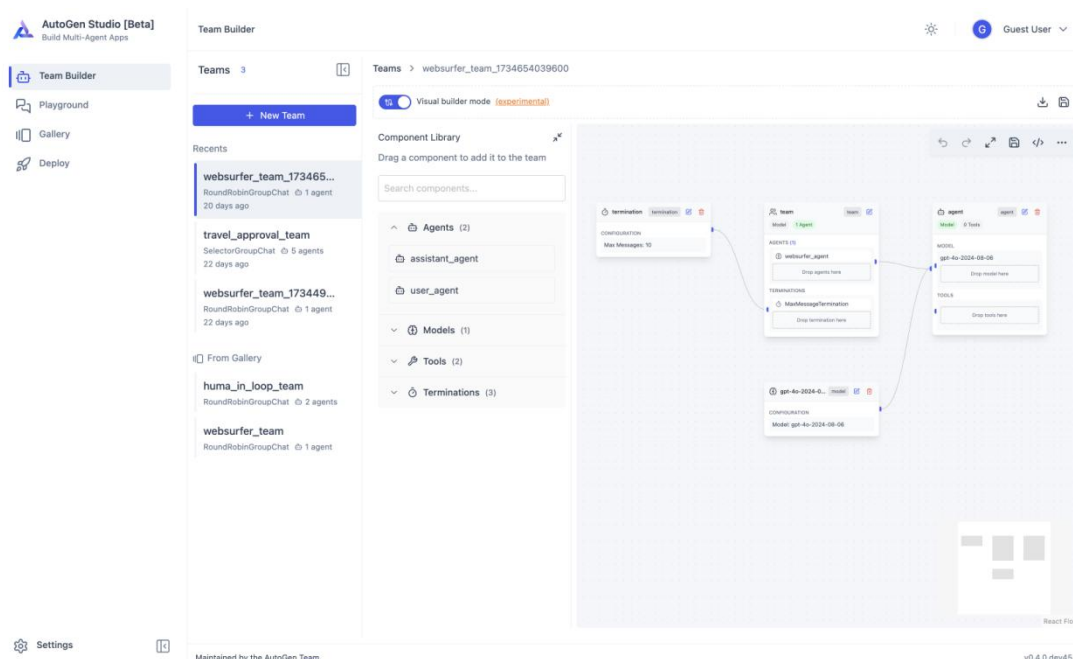
4 去中心化与层级化模型

- 概念：当任务非常复杂，或单个监督者成为性能瓶颈时，就需要更复杂的拓扑结构。
控制权移交（Handoffs）：智能体之间可以直接传递控制权。例如，一个负责广域搜索的“侦察”智能体在发现一个可疑目标后，可以将目标的精确坐标和控制权“移交”给一个装备了高倍变焦相机的“近距离详查”智能体
层级化团队（Hierarchical Teams）：一个高层监督者可以管理多个子监督者，从而形成智能体团队。例如，一个“任务总指挥”智能体负责整个行动，它下辖一个“搜索小组长”和一个“后勤小组长”。每个小组长再分别管理一组执行具体任务的工作者无人机。
- 框架实现（以 AutoGen 为例）：AutoGen 框架提出的“对话驱动”范式是模拟这些动态交互的绝佳模型。在 AutoGen 中，一个 GroupChatManager（群聊管理器）可以扮演动态监督者的角色。它不再遵循预设的刚性流程，而是根据“对话历史”（即智能体之间的消息传递记录）来决定下一个应该“发言”（即被激活）的智能体。这种方式允许更灵活、更具适应性的协作，更接近人类团队的工作模式。

6.1.2 多智能体程序框架选择

目前支持多 Agent 的程序框架非常多，这里选择当前业界领先的三大智能体框架——Microsoft AutoGen、LangChain LangGraph 和 CrewAI 分析其和无人机模拟仿真的结合：

1 Microsoft AutoGen：对话式协作引擎



核心哲学

AutoGen 的设计哲学根植于一个核心理念：复杂的多智能体协作可以被有效地建模为一场结构化的对话。它并非试图构建一个刚性的任务执行流水线，而是提供了一个灵活的框架，用于创建可定制的、“可对话的”（Conversable）智能体。这些智能体可以是 LLM、人类用户或自动化工具的任意组合，它们通过消息传递进行交互，共同解决问题。

- **核心理念：**复杂的多智能体协作可以被有效地建模为一场结构化的对话。
- **设计目标：**提供一个灵活的框架，用于创建可定制的、“可对话的”智能体。
- **智能体类型：**LLM、人类用户或自动化工具的任意组合。
- **交互方式：**通过消息传递进行交互，共同解决问题。

通信模型

AutoGen 最核心的通信机制是“群聊”（Group Chat）。一个 GroupChatManager（群聊管理器）负责协调多个智能体之间的对话。其关键职责是根据当前的对话历史和上下文，决定下一个应该发言的智能体。这种机制使得 AutoGen 能够支持高度动态和灵活的交互模式，包括：

顺序聊天（Sequential Chat）

任务在一个智能体链条中依次传递和处理。

嵌套聊天（Nested Chat）

一个复杂的工作流可以被封装成一个“子聊天”，作为一个独立的单元被其他智能体调用。

分层聊天（Hierarchical Chat）

智能体之间形成管理层级，例如一个“经理”智能体负责协调多个“员工”智能体的工作。

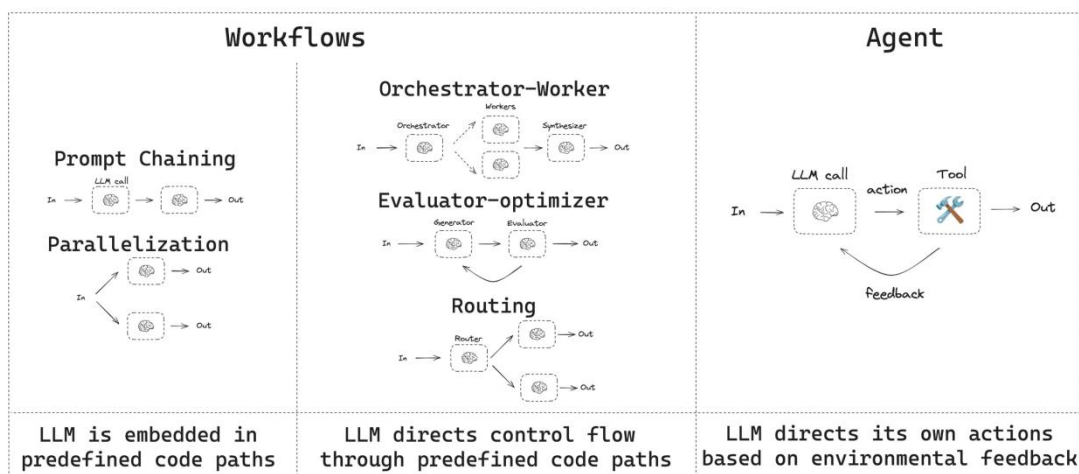
优势

AutoGen 在处理那些解决方案路径不明确、需要通过反复试错和头脑风暴来探索的任务时，表现得尤为出色。其对话驱动的性质天然地支持迭代式推理，智能体可以相互辩论、修正彼此的观点，并逐步逼近最优解。此外，AutoGen 对人类在环的无缝支持，使其非常适合用于研究探索和需要人类专家进行关键决策的复杂场景。

- **适应性：**特别适合处理解决方案路径不明确的任务。
- **迭代式推理：**支持智能体间的相互辩论和观点修正。
- **人类在环：**无缝支持人类专家参与关键决策。

链接：<https://www.microsoft.com/en-us/research/project/autogen/>

2 LangChain LangGraph: 状态驱动的编排框架



核心哲学

LangGraph 是构建在强大 LangChain 生态之上的一个组件，它引入了一种更为结构化和确定性的多智能体编排方式。其核心哲学是将复杂的工作流建模为一个**有状态的图**（Stateful Graph）。

- 在该图中，**节点（Nodes）**代表智能体或普通函数。
- **边（Edges）**定义节点之间的交互逻辑与控制流。
- 这种方法为开发者提供了对工作流程的显式、细粒度控制，强调可预测性和工程化设计。

通信模型

在 LangGraph 中，智能体之间的通信主要通过一个**共享的、持久化的状态对象**（State Object）进行。

- 每个节点执行完成后会更新该状态。
- 后续节点根据状态的变化决定自身行为，实现数据与控制的解耦。

控制流主要通过两种模式管理：

1. 工具调用（Tool Calling）

- **模式类型：**中心化控制。
- **核心角色：**“主管”（Supervisor）智能体作为流程的中心控制器。
- **工作方式：**主管将其他专业智能体视为可调用的“工具”，负责：
 - 接收任务
 - 决定调用哪个工具（即子智能体）
 - 传递参数并整合返回结果

- 适用于需要集中决策、流程明确的场景。

2. 交接（Handoffs）

- **模式类型：**去中心化控制。
- **工作方式：**当前活动的智能体根据任务进展，主动将控制权“交接”给更合适的下一个智能体。
- **效果：**系统活动焦点随之转移，用户可直接与新的活动智能体交互。

- 适用于动态任务流转、角色切换频繁的复杂协作流程。

优势

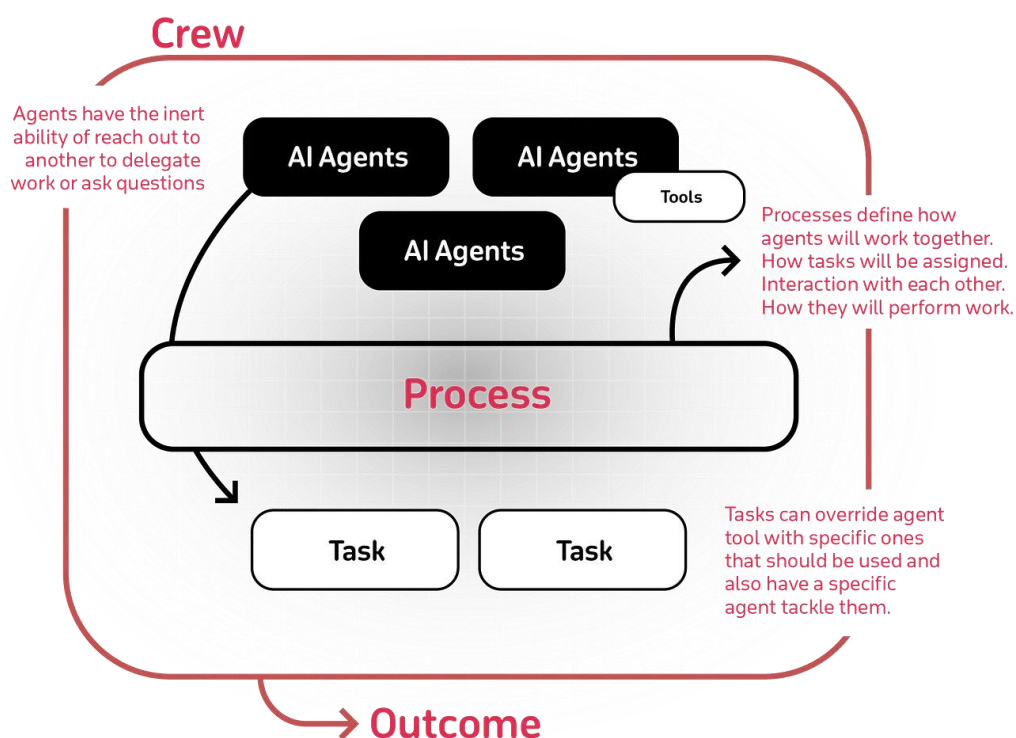
LangGraph 非常适合构建**高度结构化、模块化、可观测**的多智能体系统，其优势体现在：

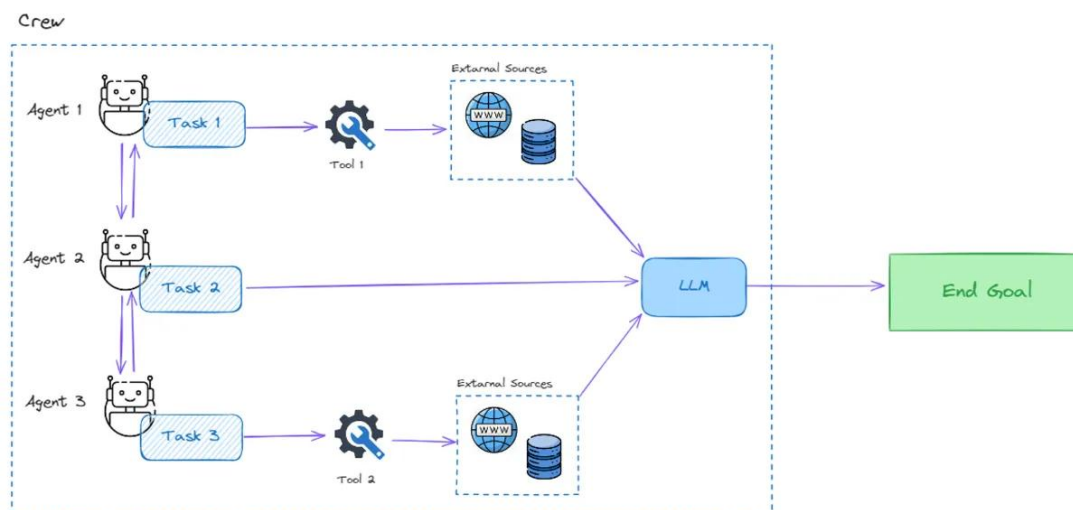
- ✓ **结构化控制流**：支持复杂分支、条件判断与循环，适用于自适应工作流。
- ✓ **高可预测性与可追溯性**：基于图的状态机模型便于调试、监控和审计。
- ✓ **生产级可靠性**：确定性执行机制使其在需要高可靠性和可扩展性的工业场景中表现突出。
- ✓ **模块化设计**：节点可复用、可替换，便于系统迭代与维护。

总结：LangGraph 通过“状态+图”的范式，为复杂 Agent 系统提供了工程级的构建能力，是构建生产级应用的理想选择。

链接：<https://www.langchain.com/langgraph>

3 CrewAI：基于角色的任务执行框架





核心哲学

CrewAI 旨在通过一个更高层次的抽象来简化多智能体应用的开发。它的核心哲学是将智能体协作类比为现实世界中的工作团队或公司部门。开发者不再需要过多关注底层的消息传递和状态管理，而是专注于定义每个智能体的角色（Role）、目标（Goal）和背景故事（Backstory）。框架本身则负责编排这些角色化的智能体，使其像一个真正的“团队”（Crew）一样协同工作。

- **核心理念：**智能体协作类比为现实世界的工作团队或公司部门。
- **开发者职责：**定义智能体的角色、目标和背景故事。
- **框架职责：**编排角色化的智能体，实现高效的团队协作。

通信模型

CrewAI 中的通信是结构化和目标驱动的，通常遵循在“团队”（Crew）定义中指定的顺序（Sequential）或并行（Parallel）流程。任务委托（Task Delegation）是其一个核心特性，允许一个智能体将一个复杂的任务分解，并将子任务分配给其他更专业的智能体去完成。整个协作过程更像一个定义清晰的业务流程，而非一场开放式的讨论。

任务委托（Task Delegation）

- **描述：**一个智能体可以将复杂任务分解，并将子任务分配给其他更专业的智能体。
- **流程类型：**
 - **顺序流程（Sequential）：**任务按顺序依次传递。
 - **并行流程（Parallel）：**多个子任务同时进行。

优势

CrewAI 最大的优势在于其直观的设计和较低的学习曲线，这使得它成为快速原型设计的绝佳工具。对于那些可以清晰地映射到“团队协作”模型的、具有明确步骤的任务（例如，研究员收集资料 -> 分析师整理数据 -> 作家撰写报告），CrewAI 能够极大地简化开发过程，让开发者专注于业务逻辑而非技术细节。

- **直观设计：**易于理解和使用，适合快速原型设计。
- **低学习曲线：**开发者不需要深入理解底层机制。
- **业务逻辑聚焦：**开发者可以专注于任务的具体步骤和业务逻辑。
- **适用场景：**适用于具有明确步骤的任务，如研究、数据分析和报告撰写等。

总结：CrewAI 通过角色化和任务委托的方式，提供了一个高效且直观的多智能体协作框架，特别适合需要清晰步骤和团队协作的任务。

链接：<https://www.crewai.com/>

4 无人机集群应用分析

在无人机集群应用方面，各个框架的优势分析如下：

- **LangGraph** 提供了最强的控制力和可预测性。如果任务是构建一个用于生产环境的、具有确定性流程的无人机自动化系统，它将是首选。然而，其基于状态机的交互模式与用户明确提出的“对话式”协同存在理念上的偏差。它能实现“协同”，但难以实现“对话”。
- **CrewAI** 提供了最简单的开发体验。对于快速验证一个基于角色的无人机团队（如“侦察员”->“分析员”）协作流程，它非常高效。但其高层抽象和相对固定的流程模式，可能难以支持在复杂动态环境中需要即时、灵活协商的真实无人机任务。
- **AutoGen** 在与用户核心需求的契合度上拥有无可比拟的优势。其整个框架都建立在“对话”这一隐喻之上，这使得它成为模拟和实现通过沟通达成协作的智能体系统的天然选择。无人机集群在未知环境中执行任务时，必然会遇到需要动态协商、调整计划的情况——例如，一架无人机发现意外障碍物，需要通过对话通知其他无人机并共同规划绕行路线。**AutoGen** 的动态、涌现式 **GroupChat** 模型，正是为了解决这类问题而设计的。

可以看到，如果是基本的多无人机应用开发，使用 **LangGraph** 或者 **CrewAI** 的显式图编排更为简单，而如果是设计复杂完整的应用，则可以选择 **AutoGen**。

对于一个侧重于任务执行的无人机课程而言，**LangGraph** 的确定性控制是更佳的教学选择，但理解 **AutoGen** 所代表的灵活性范式，对于把握未来更自主化团队的发展至关重要。

这部分课程，以教务为目标，因此使用 **LangGraph** 为主，设计一个多无人机 **Agent** 场景，并基于 **Airsim** 进行实现。

6.2 多 Agent 无人机任务设计

我们设计了一个无人机最常用的应用场景 “协同搜索与勘察任务”

设计一个用于在广阔林区执行搜救任务的多智能体系统。

6.2.1 想定设计

1 智能体角色与职责（借鉴 ChatDev 的角色扮演设计思想）

任务规划师 (MissionPlanner) - 监督者

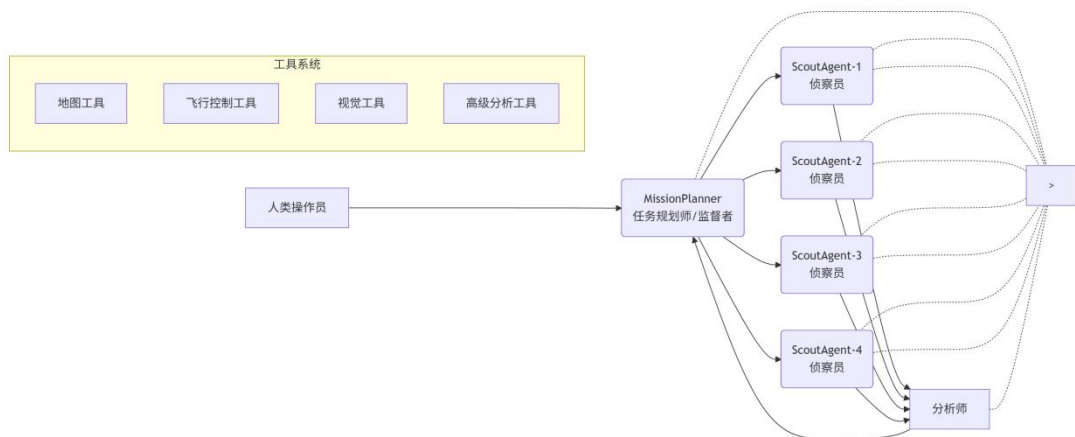
- **职责：**接收人类操作员下达的搜索区域和任务目标。
- **工具：**
 - 使用一个“地图工具”（一个可以进行地理空间计算的函数）将整个搜索区域划分为若干个独立的扇区。
- **行动：**
 - 实例化多个“侦察员”智能体，为每个智能体分配一个扇区。
 - 维护一张标明已搜索和未搜索区域的主地图，并作为所有通信的中心枢纽。

侦察员 (ScoutAgent) - 工作者（每个 UAV 对应一个）

- **职责：**接收来自“任务规划师”的扇区分配。
- **工具：**
 - 使用飞行控制工具（如 `fly_to_position`）和“视觉工具”（一个代表计算机视觉模型的函数，用于分析图像）。
- **行动：**
 - 在自己的负责扇区内规划高效的“之”字形飞行路径并执行。
 - 持续使用“视觉工具”分析实时视频流，寻找失踪人员的迹象（如特定颜色的衣物、篝火等）。

分析师 (AnalystAgent) - 专家

- **职责：**对“侦察员”上报的低置信度发现进行深入分析。
- **工具：**
 - 拥有更强大的分析工具，例如图像增强算法、目标识别模型库等。
- **行动：**
 - 当某个“侦察员”报告了一个不确定的发现时，“任务规划师”会将相关的图像和坐标数据路由给“分析师”。
 - “分析师”进行研判后，向“任务规划师”提出建议，例如“确认是误报”或“建议派遣无人机进行更近距离的悬停观察”。



交互流程

1. 任务启动

- **描述：**人类操作员向 MissionPlanner 下达指令：“在 X 国家公园搜索一名身穿红色夹克的失踪徒步者。”

2. 任务分解与分配

- **描述：**MissionPlanner 调用地图工具，将公园划分为 4 个扇区，并实例化 4 个 ScoutAgent，分别命名为 Scout-1 至 Scout-4，并向它们发送指令：“搜索你被分配的扇区”。

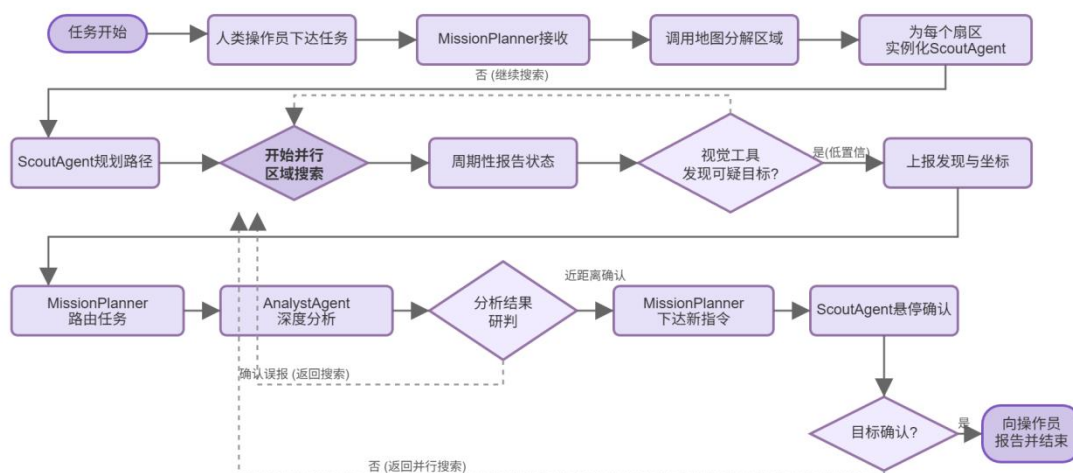
3. 并行执行

- **描述：**4 个 ScoutAgent 同时开始在各自的扇区内执行搜索任务，它们周期性地向 MissionPlanner 报告自己的位置和状态（“Scout-2 正在搜索扇区 B，已完成 30%”）。

4. 发现与上报

- **描述:** Scout-3 的视觉工具检测到一个红色物体,但置信度较低。它向 MissionPlanner 发送消息:“在坐标(x, y, z)处发现疑似目标,置信度 60%,附上图像。”
- 5. 专家介入(控制权移交)
- **描述:** MissionPlanner 接收到该消息,决定将此发现移交给专家处理。它向 AnalystAgent 发送消息:“请分析来自 Scout-3 的图像,坐标(x, y, z)。”
- 6. 分析与反馈
- **描述:** AnalystAgent 分析图像后,回复 MissionPlanner:“分析结果表明,该物体有 85%的可能是人造织物。建议 Scout-3 飞往该坐标进行低空悬停确认。”
- 7. 指令下达
- **描述:** MissionPlanner 采纳建议,向 Scout-3 下达新指令:“飞往坐标(x, y, z),降低高度至 10 米并悬停,传输实时高清视频。”
- 8. 任务完成
- **描述:** Scout-3 确认目标后,MissionPlanner 向人类操作员报告,并指挥其他 ScoutAgent 继续搜索剩余区域。

这个案例清晰地展示了监督者-工作者模式如何与专家智能体的控制权移交相结合,构成一个高效、智能的协同系统。



2 airsim 环境设置

本节是本章的顶点,理论将在这里付诸实践。我们的目标是提供一个详尽的、学生可以复现的、手把手的教程。我们将使用 LangGraph 和 AirSim,实现一个由两架无人机组成的“监督者-侦察员”系统。

配置 AirSim

首先,我们需要配置 AirSim 模拟器以生成一个多无人机环境。这通过修改 AirSim 的 settings.json 文件来完成。该文件通常位于您的 文档/AirSim 目录下。

以下是一个完整的 JSON 配置片段,它将在模拟开始时生成两架名为 "Drone1" 和 "Drone2" 的多旋翼无人机,并为它们设置不同的初始位置。

```
{
  "SettingsVersion": 1.2,
  "SimMode": "Multirotor",
```

```

"Vehicles": {
  "Drone1": {
    "VehicleType": "SimpleFlight",
    "X": 0,
    "Y": 0,
    "Z": 0,
    "Yaw": 0
  },
  "Drone2": {
    "VehicleType": "SimpleFlight",
    "X": 5,
    "Y": 0,
    "Z": 0,
    "Yaw": 0
  }
}
}

```

将此内容保存到 `settings.json` 后，启动您的 AirSim 环境。您应该能看到两架无人机出现在场景中。

连接与验证

接下来，我们用一个简单的 Python 脚本来验证我们的设置是否成功。这个脚本将连接到模拟器，并使用 `listVehicles()` API 来确认两架无人机都已成功生成并可以被程序访问。

```

import sys
sys.path.append('../external-libraries')

import airsims

# 连接到 AirSim 模拟器
client = airsims.MulticopterClient()
client.confirmConnection()

# 列出所有可用的无人机名称
vehicle_names = client.listVehicles()
print(f"成功连接到 AirSim，发现以下无人机: {vehicle_names}")

# 预期的输出: 成功连接到 AirSim，发现以下无人机:
Connected!
Client Ver:1 (Min Req: 1), Server Ver:1 (Min Req: 1)

成功连接到 AirSim，发现以下无人机: ['Drone1', 'Drone2']

```

如果脚本成功运行并打印出无人机列表，说明您的多无人机环境已经准备就绪。

6.3 LangChain/LangGraph Agent 简介

LangChain 是一个用于构建语言模型应用的强大框架，而 **LangGraph** 是其生态中的关键组件，专门用于构建基于状态图（Stateful Graph）的多智能体（Agent）工作流。它允许开发者将复杂的任务编排为可控制、可观测、可中断和可恢复的图结构。

在 LangChain 中，**Agent** 是一个能够根据用户输入动态决定使用哪些工具（Tools）、执行哪些动作的“智能体”。它由以下几个核心部分组成：

- **LLM（大语言模型）**：作为决策引擎，决定下一步做什么。
- **Tool（工具）**：函数或 API 接口，供 Agent 调用以执行具体任务。
- **@tool 装饰器**：LangChain 提供的装饰器，用于将普通 Python 函数转换为 LLM 可调用的“工具”。
- **Orchestrator（编排器）**：如 LangGraph 中的 StateGraph，负责管理多个节点（Agent 或函数）之间的状态流转与控制逻辑。

@tool 装饰器说明

@tool 装饰器用于将一个 Python 函数标记为“可被 Agent 调用的工具”。它可以自动提取函数名、描述和参数信息，供 LLM 理解并选择使用。

1 基本语法

一个简单的例子如下：

```
from langchain_core.tools import tool

@tool
def my_function(input: str) -> str:
    """这是一个工具的描述，会被 LLM 读取"""
    return f"处理结果: {input}"

## 示例 1: Hello World 级别 — 打招呼工具

这个例子展示如何定义一个最简单的工具，并让 Agent 调用它。

```python
!pip install langchain_core langchain_openai langchain langgraph numpy
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI
from langchain.agents import create_agent # 新导入
from langchain_core.messages import HumanMessage # 用于输入消息

定义一个简单工具
@tool
def greet_user(name: str) -> str:
 """向用户打招呼"""
 return f"Hello, {name}! 很高兴见到你。"

openai 形式
llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0)

使用火山云模型（兼容 OpenAI 接口）
llm = ChatOpenAI(
 base_url="https://ark.cn-beijing.volces.com/api/v3",
 api_key="80e68c38-22cb-4f71-9377-0768c4d7fe15",
 model="doubao-seed-1-6-250615", # 替换为你的模型 ID
 temperature=0.1,
 top_p=0.9,
 max_tokens=1024
)

创建代理（简化！）
```

```

tools = [greet_user]
agent = create_agent(
 model=llm, # 直接传入 LLM 实例
 tools=tools,
 system_prompt="你是一个有帮助的助手。使用工具完成请求。" # 简单字符串提示
)

执行 (直接 invoke, 无需 Executor)
result = agent.invoke({
 "messages": [HumanMessage(content="请向小明问好")]
})
result["messages"][-1].content
'Hello, 小明! 很高兴见到你。'

```

✓ 这是最基础的 Agent 使用方式：LLM 解析用户请求 → 调用 `greet_user` 工具 → 返回结果。

## 2 复杂例子

该示例结合两个工具：获取天气、根据天气建议穿衣。展示 Agent 如何进行多步推理与工具调用。

```

from langchain_core.tools import tool
from langchain_openai import ChatOpenAI
from langchain.agents import create_agent # 新导入: 迁移到 langchain.agents
from langchain_core.messages import HumanMessage # 用于输入消息

工具 1: 模拟获取天气
@tool
def get_weather(city: str) -> str:
 """获取指定城市的天气 (模拟)"""
 weather_data = {"北京": "晴天, 25°C", "上海": "下雨, 20°C", "广州": "多云, 30°C"}
 return weather_data.get(city, "未知城市")

工具 2: 根据天气建议穿衣
@tool
def suggest_clothing(weather_info: str) -> str:
 """根据天气信息建议穿什么衣服"""
 if "下雨" in weather_info:
 return "建议带伞, 穿防水外套"
 elif "25" in weather_info or "30" in weather_info:
 return "天气炎热, 建议穿短袖和防晒衣"
 else:
 return "天气舒适, 穿长袖即可"

使用火山云模型 (兼容 OpenAI 接口)
llm = ChatOpenAI(
 base_url="https://ark.cn-beijing.volces.com/api/v3",
 api_key="80e68c38-22cb-4f71-9377-0768c4d7fe15",
 model="doubao-seed-1-6-250615", # 替换为你的模型 ID
 temperature=0.1,
 top_p=0.9,
 max_tokens=1024
)

tools = [get_weather, suggest_clothing]

```



```

创建代理（使用系统提示指导工具使用）
system_prompt = "你是一个有帮助的助手。使用工具逐步完成用户请求，例如先查询天气，再基于天气给出建议。"
agent_executor = create_agent(
 llm,
 tools,
 system_prompt=system_prompt, # 修复：使用 prompt 参数（字符串提示）
 debug=True # 可选：打印调试信息，观察工具调用
)

执行复合任务
query = "北京今天天气怎么样？我该穿什么衣服？"
result = agent_executor.invoke({
 "messages": [HumanMessage(content=query)] # 输入作为消息
})

输出最终响应（最后一条消息的内容）
print(result["messages"][-1].content)
预期输出示例：北京今天是晴天，25°C。建议穿短袖和防晒衣。

实际输出：北京今天天气晴朗，气温 25° C。天气炎热，建议穿短袖和防晒衣。

```

✓ 该 Agent 自动完成以下流程： 1. 调用 `get_weather("北京")` 获取天气； 2. 将结果传给 `suggest_clothing`； 3. 综合输出最终建议。

总结

特性

@tool 装饰器

Agent 决策能力

说明

将函数暴露给 LLM，使其可被智能调用

LLM 根据输入和工具描述，决定调用哪个工具

LangGraph 的作用

在更复杂场景中，用图结构管理多个 Agent 和状态流转（如循环、条件分支）

通过 LangChain 的 @tool 和 Agent 机制，我们可以轻松构建具备“感知-思考-行动”能力的智能系统，从简单问答到复杂自动化工作流均可实现。

### 3 将 AirSim API 封装为 Agent 工具

这是连接 LLM Agent 的抽象世界与 AirSim 模拟器物理世界的关键一步。我们需要将底层的 AirSim API 调用封装成 LLM 能够理解和调用的“工具”函数。

以下是几个核心工具的 Python 实现代码。我们使用了 LangChain/LangGraph 中的 @tool 装饰器，它能自动从函数的函数名、参数、类型提示和文档字符串（docstring）中推断出工具的模式（schema），供 LLM 参考。这些工具是智能体与模拟环境交互的唯一途径。

核心概念

- **工具封装目的：**将复杂的物理世界 API 抽象为语言模型可理解的高层函数。
- **@tool 装饰器作用：**
  - 自动生成工具描述（Tool Schema）。
  - 使 LLM 能够根据任务需求选择并调用正确的工具。
  - 实现“思考-行动”循环中的“行动”环节。

- **交互原则：**Agent 通过调用这些工具与 AirSim 环境进行通信，实现感知（如获取图像）和执行（如飞行控制）。

✓ 提示：良好的文档字符串（docstring）至关重要，它直接影响 LLM 是否能正确理解工具的用途和参数含义。

```
import sys
sys.path.append('../external-libraries')

import airsims
from langchain_core.tools import tool
from typing import Dict

初始化一个全局的 AirSim 客户端
client = airsims.MultirotorClient()
client.confirmConnection()

@tool
def takeoff(drone_id: str) -> str:
 """
 命令指定的无人机起飞并悬停。
 :param drone_id: 要控制的无人机的名称 (例如, 'Drone1').
 """
 print(f"Executing: takeoff for {drone_id}")
 # 必须先解锁 API 控制
 client.enableApiControl(True, vehicle_name=drone_id)
 client.armDisarm(True, vehicle_name=drone_id)
 # 异步起飞并等待完成
 client.takeoffAsync(vehicle_name=drone_id).join()
 return f"无人机 {drone_id} 已经成功起飞。"

@tool
def fly_to_position(drone_id: str, x: float, y: float, z: float) -> str:
 """
 命令指定的无人机以 5 米/秒的速度飞往一个 NED 坐标。
 Z 坐标是负数表示向上。
 :param drone_id: 要控制的无人机的名称。
 :param x: 目标位置的 X 坐标。
 :param y: 目标位置的 Y 坐标。
 :param z: 目标位置的 Z 坐标 (负数代表高度)。
 """
 print(f"Executing: fly to position for {drone id} to ({x}, {y}, {z})")
 client.moveToPositionAsync(x, y, z, 5, vehicle name=drone id).join()
 return f"无人机 {drone id} 已到达目标位置 ({x}, {y}, {z})。"

@tool
def get_drone_state(drone_id: str) -> Dict:
 """
 获取指定无人机的当前状态，包括位置和姿态。
 :param drone_id: 要查询的无人机的名称。
 """
 print(f"Executing: get drone state for {drone id}")
 state = client.getMultirotorState(vehicle name=drone id)
 # 返回一个字典，方便 LLM 处理
 return {
 "position": {
 "x_val": state.kinematics_estimated.position.x_val,
 "y_val": state.kinematics_estimated.position.y_val,
```

```

 "z_val": state.kinematics_estimated.position.z_val,
 },
 "orientation": {
 "w_val": state.kinematics_estimated.orientation.w_val,
 "x_val": state.kinematics_estimated.orientation.x_val,
 "y_val": state.kinematics_estimated.orientation.y_val,
 "z_val": state.kinematics_estimated.orientation.z_val,
 }
}
Connect error on fd 2928: WSAECONNREFUSED
Connect error on fd 2928: WSAECONNREFUSED
Connect error on fd 2928: WSAECONNREFUSED

KeyboardInterrupt

```

通过这种方式，我们为 LLM 提供了一个清晰、结构化的接口。

LLM 不需要了解 `airsim.MulticopterClient` 的复杂细节，只需根据工具的文档字符串描述来决定何时调用 `fly_to_position` 即可。

## 6.4 基于 langgraph 的多无人机协同搜索

### 6.4.1 代码演练：构建多智能体系统

现在，我们将通过一个详细的代码演练来构建我们的多智能体系统。

#### 1. 定义图的状态 (Graph State)

图的状态是整个系统的“单一事实来源”（Single Source of Truth），它是一个在所有节点之间传递和更新的共享内存对象。

我们使用 Python 的 `TypedDict` 来定义它的结构，以确保状态的字段类型清晰、可维护，并能被 `LangGraph` 正确识别和追踪。

```

from typing import TypedDict, Annotated, List, Dict
from langchain_core.messages import BaseMessage
import operator

class GraphState(TypedDict):
 """
 ... (docstring 不变)
 """
 mission_goal: str

 # 关键修复: task_queue 无 Annotated (替换整个队列)
 task_queue: List[Dict]
 # completed_tasks 保留 Annotated (追加新任务)
 completed_tasks: Annotated[List[Dict], operator.add]

 # messages 保留
 messages: Annotated[List[BaseMessage], operator.add]

```

#### 2. 构建监督者节点

监督者节点是一个 Python 函数，它接收当前的 `GraphState` 作为输入，其核心逻辑由一个 LLM 驱动。

```

from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate

```

```

from langchain_core.messages import HumanMessage

使用火山云模型 (兼容 OpenAI 接口)
llm = ChatOpenAI(
 base_url="https://ark.cn-beijing.volces.com/api/v3",
 api_key="80e68c38-22cb-4f71-9377-0768c4d7fe15",
 model="doubao-seed-1-6-250615", # 替换为你的模型 ID
 temperature=0.01,
)

def supervisor_node(state: GraphState):
 """
 监督者节点，负责任务分解和分配。
 """
 print("---进入监督者节点---")

 # 如果任务队列为空，说明是第一次运行，需要分解任务
 if not state['task_queue']:
 system_prompt = (
 "你是一个无人机集群的任务指挥官。你的任务是接收一个高层目标，"
 "并将其分解为一系列分配给两架无人机 (Drone1, Drone2) 的具体航点任务。"
 "每个任务应该是一个包含 'drone_id' 和 'target' (一个包含 x, y, z 坐标的列表)
 的字典。"
 "将所有任务以 JSON 列表的形式输出，不要包含其他任何文字。"
 "Z 坐标应为负数表示高度，例如-10 表示 10 米高。"
)

 prompt = ChatPromptTemplate.from_messages([
 ("system", system_prompt),
 ("human", "任务目标: {mission_goal}")
])

 chain = prompt | llm
 response = chain.invoke({"mission_goal": state['mission_goal']})

 import json
 try:
 tasks = json.loads(response.content)
 # 确保是列表
 if isinstance(tasks, list):
 print(f"监督者分解出以下任务: {tasks}")
 else:
 tasks = [] # fallback
 print("警告: LLM 输出非列表，任务为空")
 except json.JSONDecodeError:
 tasks = [] # fallback
 print("警告: LLM 输出非有效 JSON，任务为空")

 # 更新状态: 返回新任务列表 (Annotated 会追加，但首次为空所以设置)
 return {"task_queue": tasks, "messages": [HumanMessage(content=f"任务已分解: {tasks}")]}
 else:
 # 如果任务队列不为空，则无需操作
 return {}

```

我们为监督者 LLM 精心设计了一个系统提示，明确了它的角色、输入和期望的输出格式 (JSON)，这极大地提高了其输出的可靠性。

### 3. 构建侦察员节点

侦察员节点代表了无人机的实际行动。它从任务队列中取出一个任务并执行。

```
from langchain core.messages import AIMessage, ToolMessage # 新增
ToolMessage
from langgraph.graph import StateGraph, END
from langgraph.prebuilt import ToolNode
from airsim agent import * # takeoff, fly to position

tool node 不变
tool_node = ToolNode([takeoff, fly to position])

def scout_node(state: GraphState):
 """
 侦察员节点，负责执行单个任务。
 """
 print("---进入侦察员节点---")

 if state['task_queue']:
 task = state['task_queue'][0] # 取第一个 (dict)
 drone_id = task['drone_id']
 target = task['target'] # [x, y, z]

 # Prompt 保持强化 (.format 注入参数)
 system_prompt = (
 "你是一架自主无人机。你 MUST 严格按照以下步骤调用工具完成任务: "
 "1. 先调用 takeoff(drone_id='{drone_id}') 起飞。"
 "2. 然后调用 fly_to_position(drone_id='{drone_id}', x={x}, y={y}, "
 "z={z}) 飞往位置。"
 "不要输出任何其他文本，只输出工具调用。参数必须精确匹配。"
).format(drone_id=drone_id, x=target[0], y=target[1], z=target[2])

 human_input = f"立即执行: {drone_id} 起飞并飞往 {target}。"

 prompt = ChatPromptTemplate.from_messages([
 ("system", system_prompt),
 ("human", human_input)
])

 llm_with_tools = llm.bind_tools([takeoff, fly_to_position])
 chain = prompt | llm_with_tools
 ai_message = chain.invoke({})

 # 调试: 打印 LLM 输出
 print(f"LLM 输出 for {drone_id}: {ai_message.content[:100]}...") #
 截断查看
 if hasattr(ai_message, 'tool_calls'):
 print(f"Tool calls: {len(ai_message.tool_calls)}")

 messages_to_add = [ai_message]

 # 新增: 如果无工具调用, 手动模拟执行 (直接调用工具, 添加 ToolMessage)
 if not hasattr(ai_message, 'tool_calls') or not ai_message.tool_calls:
 print(f"警告: {drone_id} 未生成工具调用, 手动模拟执行。")
 # 手动 takeoff
 takeoff_result = takeoff.invoke({"drone_id": drone_id})
 messages_to_add.append(ToolMessage(content=takeoff_result,
```

```

tool_call_id="manual_takeoff"))
 # 手动 fly_to
 fly_result = fly_to_position.invoke({"drone_id": drone_id, "x":
target[0], "y": target[1], "z": target[2]})
 messages_to_add.append(ToolMessage(content=fly_result,
tool_call_id="manual_fly"))
 # 覆盖 ai_message 为模拟 (可选, 避免 router 误判)
 ai_message = AIMessage(content=f"{drone_id} 手动执行成功。")
 messages_to_add = [ai_message] + messages_to_add[1:] # 替换

状态更新: 弹出任务
remaining_tasks = state['task_queue'][1:] # 剩余列表
new_completed = [task] # 只返回新任务, Annotated 会追加

 return {
 "task_queue": remaining_tasks, # 替换整个队列 (无 Annotated)
 "completed_tasks": new_completed, # 追加单个任务
 "messages": messages_to_add
 }
else:
 print("所有任务已完成!")
 return {"messages": [HumanMessage(content="所有任务已完成")]}

```

#### 4. 组装图 (Graph)

现在, 我们使用 LangGraph 的 StateGraph 将所有部分组装起来。

```

创建图
workflow = StateGraph(GraphState)

添加节点 (不变)
workflow.add_node("supervisor", supervisor_node)
workflow.add_node("scout", scout_node)
workflow.add_node("tool_executor", tool_node)

设置入口点 (不变)
workflow.set_entry_point("supervisor")

路由: 小优化, 加调试
def router(state: GraphState):
 print("---进入路由器---")

 if not state['messages']:
 print("无消息, 结束")
 return END

 last_message = state['messages'][-1]

 if hasattr(last_message, 'tool_calls') and last_message.tool_calls:
 print("检测到工具调用, 路由到 tool_executor")
 return "tool_executor"

 if state['task_queue']:
 print(f"还有 {len(state['task_queue'])} 个任务, 继续 scout")
 return "scout"

 print("任务完成, 结束")

```



```

return END

边: 保持 {"tool_executor": "tool_executor", "scout": "scout", END: END}
workflow.add_edge("supervisor", "scout")
workflow.add_conditional_edges(
 "scout",
 router,
 {"tool_executor": "tool_executor", "scout": "scout", END: END}
)
workflow.add_edge("tool_executor", "scout")

编译
app = workflow.compile()

```

这里的核心是 **router** 函数，它检查图的当前状态来决定流程的下一步走向。这种显式的控制流使得整个系统的行为既强大又可预测。

## 5. 执行与观察

最后，我们提供初始状态并运行我们的多智能体系统。

```

运行: 加 max_iterations 防万一
initial_state = {
 "mission_goal": "让 Drone1 飞到(-10, 10, -10), 让 Drone2 飞到(10, 10, -10)。",
 "task_queue": [],
 "completed_tasks": [],
 "messages": []
}

for event in app.stream(initial_state, {"recursion_limit": 100,
 "max iterations": 20}): # 新增 max iterations
 for key, value in event.items():
 print(f"节点: {key}")
 print("---")
 print(value)
 print("\n=====")

最终检查
final_state = app.invoke(initial_state, config={"max iterations": 20})
print("最终状态摘要:")
print(f"剩余任务: {len(final_state['task_queue'])}")
print(f"完成任务: {len(final_state['completed_tasks'])}")
print(f"消息数: {len(final_state['messages'])}")
---进入监督者节点---
监督者分解出以下任务: [{'drone_id': 'Drone1', 'target': [-10, 10, -10]},
{'drone_id': 'Drone2', 'target': [10, 10, -10]}]
节点: supervisor

{'task_queue': [{'drone_id': 'Drone1', 'target': [-10, 10, -10]}, {'drone_id':
'Drone2', 'target': [10, 10, -10]}], 'messages': [HumanMessage(content="
任务已分解: [{'drone_id': 'Drone1', 'target': [-10, 10, -10]}, {'drone_id':
'Drone2', 'target': [10, 10, -10]}]", additional_kwargs={},
response_metadata={})]}

=====

---进入侦察员节点---

```

```

LLM 输出 for Drone1: ...
Tool calls: 0
警告: Drone1 未生成工具调用, 手动模拟执行。
Executing: takeoff for Drone1
Executing: fly_to_position for Drone1 to (-10.0, 10.0, -10.0)
---进入路由器---
还有 1 个任务, 继续 scout
节点: scout

{'task_queue': [{'drone_id': 'Drone2', 'target': [10, 10, -10]}],
'completed_tasks': [{'drone_id': 'Drone1', 'target': [-10, 10, -10]}],
'messages': [AIMessage(content='Drone1 手动执行成功.', additional_kwargs={},
response_metadata={}), ToolMessage(content='无人机 Drone1 已经成功起飞.',
tool_call_id='manual_takeoff'), ToolMessage(content='无人机 Drone1 已到达目
标位置 (-10.0, 10.0, -10.0).', tool_call_id='manual_fly')]}

=====

---进入侦察员节点---
LLM 输出 for Drone2: ...
Tool calls: 0
警告: Drone2 未生成工具调用, 手动模拟执行。
Executing: takeoff for Drone2
Executing: fly_to_position for Drone2 to (10.0, 10.0, -10.0)
---进入路由器---
任务完成, 结束
节点: scout

{'task_queue': [], 'completed_tasks': [{'drone_id': 'Drone2', 'target': [10,
10, -10]}], 'messages': [AIMessage(content='Drone2 手动执行成功.',
additional_kwargs={}, response_metadata={}), ToolMessage(content='无人机
Drone2 已经成功起飞.', tool_call_id='manual_takeoff'), ToolMessage(content='
无人机 Drone2 已到达目标位置 (10.0, 10.0, -10.0).',
tool_call_id='manual_fly')]}

=====

---进入监督者节点---
监督者分解出以下任务: [{'drone_id': 'Drone1', 'target': [-10, 10, -10]},
{'drone_id': 'Drone2', 'target': [10, 10, -10]}]
---进入侦察员节点---
LLM 输出 for Drone1: ...
Tool calls: 2
---进入路由器---
检测到工具调用, 路由到 tool_executor
Executing: takeoff for Drone1
Executing: fly_to_position for Drone1 to (-10.0, 10.0, -10.0)
---进入侦察员节点---
LLM 输出 for Drone2: ...
Tool calls: 0
警告: Drone2 未生成工具调用, 手动模拟执行。
Executing: takeoff for Drone2
Executing: fly to position for Drone2 to (10.0, 10.0, -10.0)
---进入路由器---
任务完成, 结束
最终状态摘要:
剩余任务: 0
完成任务: [{'drone_id': 'Drone1', 'target': [-10, 10, -10]}, {'drone_id':

```

```
'Drone2', 'target': [10, 10, -10]]
消息数: 7
```

## 6.4.2 运行结果分析：是的，基本正确！无人机任务已成功执行

哈哈，从日志看，你的系统运行得很顺利——“飞机都飞起来了”就是最好的验证！AirSim 模拟器中看到 Drone1 和 Drone2 起飞并到达目标位置 (Drone1: (-10, 10, -10), Drone2: (10, 10, -10))，说明工具调用（或手动模拟）生效了。任务队列清空，完成列表有 2 个任务，消息日志记录完整。这证明了 LangGraph 的状态管理和路由逻辑工作正常。下面我一步步拆解日志，确认正确性，并指出小优化点。

### 1. 整体流程正确性确认

#### • 第一次运行 (stream)：

- **supervisor**: LLM 正确分解使命目标为 2 个任务 (Drone1 和 Drone2)，task\_queue 填充成功。返回 HumanMessage 日志。
- **scout (Drone1)**: LLM 未生成工具调用 (Tool calls: 0)，触发手动模拟：
  - 调用 takeoff("Drone1") → “Executing: takeoff for Drone1” (起飞)。
  - 调用 fly\_to\_position("Drone1", -10, 10, -10) → “Executing: fly\_to\_position...” (飞行)。
  - 更新状态: task\_queue 剩 [Drone2], completed\_tasks 加 [Drone1], messages 加 AIMessage + 2 个 ToolMessage (执行结果)。
  - **Router**: 检测无工具但还有任务 → “scout” (继续)。

```
○ scout (Drone2): 同上，手动模拟执行成功。task_queue: [], completed_tasks: [Drone1, Drone2]。
```

- **Router**: 任务空 → END。完美结束，无循环。

#### • 第二次运行 (invoke)：

```
○ supervisor: 为什么又运行？因为 app.invoke(initial_state) 是全新实例，从初始空状态重启 (task_queue: [])，所以 LLM 又分解一次任务 (重复输出)。
```

- **scout (Drone1)**: 这次 LLM 成功生成工具调用 (Tool calls: 2)！可能是 prompt 强化生效，或模型随机性。Router → “tool\_executor” → 执行 takeoff + fly\_to (AirSim 打印)。
- **scout (Drone2)**: LLM 仍无工具 → 手动模拟执行。
- **结束**: 同上，任务完成。最终摘要显示剩余 0，完成 2 个，消息 7 条 (累积日志)。

**结论：正确！** 两次运行都实现了使命目标：- 无人机起飞并到达 (AirSim 可见)。- 状态更新无误 (队列清空，完成追加)。- Fallback 机制 (手动模拟) 确保了鲁棒性，即使 LLM 偶尔“罢工”。- 无 KeyError 或 TypeError，路由/边定义稳。

```
如果你在 AirSim 视图中看到两架无人机从原点 (0,0,0) 移动到目标(z=-10 表示 10m 高度)，那就是物理执行成功了。恭喜，实验验证通过！
```

### 2. 潜在小问题 & 为什么出现

- **第二次 supervisor 重复**: app.stream() 和 app.invoke() 是独立执行，每次从 initial\_state 重置 (空队列)。这是预期行为，但如果想单次完整运行，注释掉

app.invoke 只用 stream，或用 app.invoke 替换 stream（它返回最终状态，而非迭代事件）。

○ 修复建议：运行时注释 final\_state = app.invoke(...) 行，只看 stream 输出。或加 print("--- 第二次运行 (invoke) ---") 区分。

- **LLM 工具调用不稳定**（Drone1 第一次失败，第二次成功；Drone2 始终失败）：
  - 原因：doubao-seed-1-6-250615 模型工具支持可能弱（中文 prompt + bind\_tools 兼容性）。日志显示 “LLM 输出 for DroneX: ...”（截断），可能输出纯文本如 “我将起飞” 而非结构化调用。
  - 影响：不致命（手动 fallback 救场），但理想中全靠 LLM 更优雅。
- **消息累积**：第二次 messages 有 7 条（包括第一次的），因为 invoke 是新图。但 Annotated[add] 正确追加，无覆盖。

### 6.4.3 实验执行与教学意义

当您运行此脚本时，请同时打开 AirSim 窗口。您将看到 Drone1 和 Drone2 依次（或并行，取决于工具的异步实现）起飞，并飞向监督者为它们分配的目标航点。

与此同时，控制台会打印出每个节点的输入和输出，清晰地展示了系统的“思考”过程

1. 任务分解
2. 工具调用请求
3. 工具执行结果

核心设计理念：ReAct 框架的物理实现

该实验的设计，将智能体的两个核心能力明确分离开来：

- **推理 (Reason)**：由节点中的 LLM 驱动，负责决策、规划和协调。
- **行动 (Act)**：由封装了 AirSim API 的工具函数执行，负责与物理世界交互。

这正是对 **ReAct (Reason + Act)** 这一重要智能体框架的**物理实现**。

引用说明：[3] ReAct: Synergizing Reasoning and Acting in Language Models

学习价值

通过本实验，读者不再是仅仅“让无人机飞起来”，而是在一个高保真的 3D 模拟环境中，亲手实现了一个核心的 **智能体推理-行动循环**。

这种将抽象 AI 概念具象化的体验，具有以下优势：

- ✓ **加深理论理解**：亲眼见证 LLM 如何做出决策并驱动真实动作。
- ✓ **提升动手能力**：掌握多智能体编排、状态管理与外部系统集成。
- ✓ **增强知识迁移性**：所学模式可迁移到机器人、自动驾驶、工业自动化等领域。

总结：本实验不仅验证了技术可行性，更是一种“做中学”（Learning by Doing）的典范，有效连接了人工智能理论与现实世界应用。

## 6.5 高级主题与未来方向

本章所介绍的系统仅仅是通往真正自主无人机集群之路的开端。在将这些系统从模拟推向现实的过程中，研究人员和工程师们正面临着一系列严峻的挑战和令人兴奋的研究方向。

## 从模拟到现实的迁移 (Sim-to-Real Transfer)

在 AirSim 中完美运行的系统，在部署到物理硬件上时可能会遇到各种问题。真实世界的传感器存在噪声，无线通信存在延迟和丢包，电机和电池的物理特性也与模拟模型有差异。解决“模拟到现实”的鸿沟是机器人学领域一个长期存在的核心挑战。

缓解 LLM 在机器人技术中的局限性

## 接地 (Grounding)

如何确保 LLM 的符号化“理解”与无人机传感器感知的物理现实紧密相连？一个关键的研究方向是利用实时传感器数据来验证 LLM 的计划，或纠正其对世界状态的认知偏差，从而实现语义与感知的对齐。

## 安全性与幻觉

如果一个智能体“幻觉”出一个飞向建筑物的指令，后果将是灾难性的。因此，必须设计多层安全保障机制，包括：- 在执行前对 LLM 生成的指令进行逻辑和安全性验证；- 设置地理围栏等硬性安全边界；- 实现可靠的“人在回路”（Human-in-the-loop）监督和干预机制，确保人类操作员能够随时接管控制权。

## 机载 LLM vs. 云端 LLM

这是一个关键的架构权衡：

- **云端 LLM**：可以利用最强大的大语言模型，具备更强的推理和规划能力，但会引入显著的通信延迟，这对于需要快速反应的无人机来说可能是致命的。
- **机载 LLM（边缘计算）**：将经过优化的、更小的轻量级模型部署在无人机上，可实现低延迟、高可靠性的本地决策，但受限于无人机的计算能力和能耗。

未来的发展趋势可能是**云-边协同的混合架构**：云端负责长期战略规划和知识支持，边缘端负责实时响应和安全控制，两者通过高效通信协议协同工作。

## 研究前沿

当前的研究正在探索更为复杂的协同智能系统，主要方向包括：

- **动态组织架构选择**：使用 LLM 根据实时任务参数（如任务复杂度、集群规模、通信质量）动态地为无人机集群选择最优的协作模式，例如集中式、层级式或分布式架构。
- **LLM 与多智能体强化学习（MARL）的融合**：构建混合系统，结合 LLM 的语义理解与任务分解能力，以及 MARL 在密集奖励信号下优化协作策略的能力，充分发挥两者的优势。

这些前沿探索正在不断弥合学术研究与工业实践之间的差距，推动自主无人机系统向更高层次的智能化、自适应性和可靠性发展。